Clinostato

TRABAJO FINAL — CLINOSTATO DE DOS EJES

PROFESORES TITULARES: ING. DANIEL GERMENA – ING. LUCIANO MARGARIA
ALUMNOS: GARELLI VALENTÍN, MAGGI TOMÁS, RONDÓN JUAN IGNACIO

Índice

1. Introducción....3
1.1. ¿Qué es un clinostato?....3
1.2. Motivación....4
1.3. Problema y necesidad....4
1.4. Objetivo de este informe....4
2. Relevamiento del mercado (Benchmark)....4
2.1. Soluciones comerciales....4
2.2. Clinostato open-source....4
3. Requisitos y restricciones....5
3.1. Requisitos funcionales....5
3.2. Requisitos no funcionales....6
3.3. Restricciones....6
3.4. Supuestos....6
4. Metodología de trabajo....6
4.1. Enfoque adoptado....6
4.2. Etapas del proyecto....6
4.3. Cambios introducidos durante el desarrollo....7
4.4. Decisión técnica: motores DC vs. paso a paso....8
5. Alcance de la prueba de concepto....8
5.1. Criterios de aceptación....8
5.2. Entregables....8
5.3. Hitos....8
6. Distribución de tareas....8
6.1. Bloques de trabajo....8
6.2. Matriz de responsabilidades....9
6.3. Dependencias entre bloques....9
7. Arquitectura del sistema....9
7.1. Diagrama de bloques general....9
7.2. Interfaces....10
8. Componentes principales....10
8.1. Microcontrolador: STM32 Nucleo-G491RE....10
8.2. Cuerpo y estructura....11
8.3. Motores DC JGA25-370....11
8.4. Sensado de estado: encoder óptico....12
9. Descripción técnica del sistema de control....13
9.1. Actuación: control del motor por PWM....13
9.2. Sensado: medición de velocidad con timer....14
9.3. Lazo de control PID....14
10. Desarrollo de las tareas por bloque....14
10.1. Bloque 1: Cuerpo del clinostato....14

10.1.1. Modificación del cuerpo para los nuevos motores..... 14
1. Rediseño de la base:..... 15
2. Aplanar la base:..... 15
3. Adaptar el motor:..... 16
10.1.2. Modificación del cuerpo para el sensor..... 17
10.1.3. Elevación del cuerpo..... 17
10.2. Bloque 2: Firmware..... 18
10.2.1. Movimiento de los motores con PWM..... 18
10.2.2. Sensado con el encoder mediante Input Capture..... 18
10.2.3. Implementación del control PID..... 19
10.3. Bloque 3: Hardware..... 19
10.3.1. Adaptación de alimentación del sensor..... 19
10.3.2. Conexión del driver del motor (puente H)..... 20
10.4. Bloque 4: Integración total..... 20
10.4.1. Armado del cuerpo..... 20
10.4.2. Funcionamiento del lazo PID..... 21
11. Resultados..... 22

1. Introducción

1.1. ¿Qué es un clinostato?

El clinostato es un dispositivo utilizado para simular los efectos de la microgravedad, aplicado principalmente en estudios de cultivos de plantas y de microorganismos. La técnica consiste en someter a la muestra, ubicada en el centro del dispositivo, a una rotación aleatoria en dos ejes. La aleatoriedad es una condición necesaria: el movimiento no debe presentar patrones que la muestra pueda percibir, de modo que el vector de gravedad promediado en el tiempo tienda a cero.



Figura 1. *Ejemplo de clinostato de dos ejes.*

1.2. Motivación

La elección de este proyecto responde a tres motivos principales:

- **Como tesis:** requiere la aplicación integrada de conceptos de teoría de control, sistemas embebidos, electrónica de potencia y adquisición de datos.
- **Utilidad académica con baja inversión:** es un dispositivo ampliamente utilizado en el ámbito académico y experimental, principalmente en carreras vinculadas a la biología y la química.
- **Interdisciplinariedad:** el proyecto se nutre de la colaboración con otras disciplinas, como agronomía, ciencias químicas y ciencias de la salud.

1.3. Problema y necesidad

- **Problema:** la imposibilidad de realizar ensayos en condiciones reales de microgravedad.
- **Necesidad:** contar con la capacidad de realizar pruebas en microgravedad simulada a bajo costo y con alta accesibilidad.

1.4. Objetivo de este informe

Este informe documenta la etapa de prueba de concepto del clinostato, correspondiente al cierre de las materias: “Proyecto y Diseño” y “Teoría de Control”. En primer lugar se presenta el enfoque metodológico adoptado para gestionar el proyecto y los ajustes que fue necesario introducir a medida que se avanzó. Luego se detalla cómo se organizó el trabajo en cuatro bloques principales y cómo se distribuyeron entre los integrantes del equipo. Finalmente, se describe el desarrollo técnico de cada bloque.

2. Relevamiento del mercado (Benchmark)

Se relevaron las alternativas disponibles actualmente para identificar sus limitaciones y definir el aporte del proyecto.

2.1. Soluciones comerciales

Existen equipos comerciales, como los ofrecidos por **CelVivo**, que resuelven el problema pero presentan desventajas para el contexto académico local: un precio elevado y una arquitectura cerrada que no permite modificaciones ni acceso a los datos internos.

2.2. Clinostato open-source

Como alternativa existe un diseño de código abierto, imprimible en 3D, publicado en un repositorio público (**github.com/wahabk/Clinostat**). Si bien resuelve el problema del costo y la accesibilidad, carece de sensado del estado del dispositivo y su movimiento está precargado, sin posibilidad de verificar ni ajustar la aleatoriedad en tiempo real.

Tabla 1. Comparación de las alternativas relevadas.

Solución	Ventajas	Limitaciones
Comercial (CelVivo)	Producto terminado y validado	Precio elevado, sistema cerrado, dependencia de WiFi propietario
Open-source	Bajo costo, imprimible en 3D	Sin sensado, movimiento precargado
Propuesta del proyecto	Bajo costo, abierto, con sensado y control	A desarrollar y validar

3. Requisitos y restricciones

3.1. Requisitos funcionales

- Regular la velocidad de rotación.
- Sensar las variables de estado del dispositivo.
- Generar movimiento aleatorio.
- Visualizar los datos adquiridos.
- Contar con alimentación eléctrica

.

3.2. Requisitos no funcionales

- Operación sobre una red doméstica (WiFi).
- Uso de un microcontrolador STM32.
- Actuación mediante motores eléctricos con control de velocidad (originalmente se planteó el uso de motores paso a paso; ver sección 4.3).
- Sensado del estado del dispositivo (originalmente se planteó una IMU; ver sección 4.3).

3.3. Restricciones

- Funcionamiento ininterrumpido durante las pruebas.
- Movimiento no repetitivo.
- Espacio físico disponible.

3.4. Supuestos

- Alimentación eléctrica constante.
- Disponibilidad de conectividad WiFi.
- Conocimientos básicos de operación por parte del usuario.
- Disponibilidad de una computadora capaz de comunicarse con el sistema.

4. Metodología de trabajo

4.1. Enfoque adoptado

Para la gestión del proyecto se adoptó una **metodología de carácter estático**, de tipo secuencial o en cascada. Bajo este enfoque, el trabajo se **organiza en etapas que se ejecutan en un orden predefinido**: primero se fijan los requisitos, luego se diseña la solución, se adquieren los componentes, se construye y finalmente se integra y se valida. Cada etapa toma como entrada los resultados de la anterior y el alcance se define al comienzo del proyecto.

La elección de este enfoque se **justifica por las características del trabajo**: el alcance de la prueba de concepto quedó acotado mediante los criterios de aceptación acordados con los docentes, las dependencias entre tareas son fuertes (no es posible integrar el lazo de control sin contar con el cuerpo, la etapa de potencia y el sensado).

4.2. Etapas del proyecto

El desarrollo general se organiza en las siguientes etapas:

- **Estudio principal:** primer acercamiento al funcionamiento del dispositivo y relevamiento de alternativas existentes.
- **Requisitos:** análisis de los requisitos y limitaciones del dispositivo; búsqueda y adquisición de componentes.
- **Diseño:** definición de la arquitectura del sistema y división del trabajo en bloques.

- **Construcción:** desarrollo en paralelo de cada bloque (cuerpo, firmware y hardware).
- **Prueba de concepto:** integración del prototipo y prueba de funcionamiento con control PID en uno de sus ejes. Esta etapa cierra las materias Proyecto y Diseño y Teoría de Control.

La hoja de ruta agrupa estas etapas en tres grandes bloques: planificación (investigación general, definición y compra de componentes), construcción y movimiento (armado del cuerpo y movimiento).

4.3. Cambios introducidos durante el desarrollo

Si bien el proyecto se planificó con un enfoque estático, a medida que se avanzó surgieron problemas que obligaron a revisar algunas decisiones tomadas en las etapas iniciales. Estos cambios se incorporaron. Los principales fueron:

Tabla 2. Cambios respecto de la planificación original.

Planificación original	Cambio realizado	Motivo
Motores paso a paso (NEMA 17)	Motores DC JGA25-370 con reductora	Mayor linealidad y simplificación para obtener modelos precisos aplicables a la simulación y al diseño del controlador.
Sensado de estado mediante IMU	Encoder óptico de ranura sobre el eje del motor	Para el control de velocidad de un eje se necesita una medición directa y confiable de la velocidad del motor.
Estructura original del repositorio sin modificar	Cuerpo modificado para los nuevos motores y el sensor	El diseño original está pensado para motores paso a paso; el cambio de actuador requirió rediseñar los soportes.
Control de los dos ejes	Control de un único eje en esta etapa	Alcance acordado con los docentes para la prueba de concepto; el segundo eje queda para la tesis.
Falta de correas de 500mm	Compra de 1m y adaptación casera	La falta de stock imposibilita la compra de la correa de 500mm necesaria para el movimiento de uno de los ejes, por ello, se optó por comprar, cortar y unir una correa de mayor tamaño

4.4. Decisión técnica: motores DC vs. paso a paso

Se evaluaron dos alternativas de actuación. Si bien los motores DC ofrecen menor complejidad matemática, requirieron rediseñar la estructura. Los motores paso a paso se adaptan directamente al diseño elegido; sin embargo, en pos de un mayor alcance a futuro, se optó por los motores DC.

Tabla 3. Comparación de alternativas de actuación.

Criterio	Motores DC	Motores paso a paso
Linealidad	Movimiento lineal	Movimiento no lineal
Forma de control	Etapa de potencia (puente H)	Drivers dedicados
Precio	Medio	Medio
Complejidad matemática	Reducida	Media-alta
Complejidad de estructura	Media (requiere rediseño)	Baja (ideal con el diseño actual)

5. Alcance de la prueba de concepto

5.1. Criterios de aceptación

Durante el transcurso de la cursada se definieron, junto con el docente, los requisitos del proyecto para la aprobación de las materias:

- Armado del cuerpo.
- Sensado de uno de los motores.
- Aplicación de un lazo de control al motor sensado.

5.2. Entregables

- Cuerpo del clinostato armado.
- Movimiento de uno de los ejes controlado.
- Sensado de la velocidad recibido por el microcontrolador.

5.3. Hitos

1. Diseño del cuerpo evaluado y aceptado.
2. Ensamblado.
3. Lograr un movimiento fluido.

Dado que la prueba de concepto abarca un solo eje, las modificaciones realizadas sobre el cuerpo se limitaron a lo **necesario para ese eje**. El resto de la estructura queda planteada para avanzar en el futuro con la tesis.

6. Distribución de tareas

6.1. Bloques de trabajo

Para la prueba de concepto, el trabajo se **dividió en cuatro grandes bloques**. Cada bloque tiene un responsable principal, encargado de coordinarlo y de garantizar que se cumplan sus objetivos. Sin embargo, **todos los integrantes participaron en todos los bloques** y colaboraron en el desarrollo de las tareas de los demás, lo que permitió que el equipo completo conozca el sistema de punta a punta y facilitó la integración final.

Tabla 4. Bloques de trabajo, tareas y responsables.

Bloque	Tareas principales	Responsable principal
1. Cuerpo del clinostato	Modificar el cuerpo para el sensor. Modificar el cuerpo para que quepan los nuevos motores. Levantar el cuerpo.	Juan Rondón
2. Firmware	Mover los motores mediante PWM. Sensar la velocidad con el encoder mediante Input Capture. Aplicar el control PID.	Valentín Garelli
3. Hardware	Adaptar la alimentación del sensor. Conexionado del driver del motor (puente H).	Tomás Maggi
4. Integración total	Armado del cuerpo con motores, sensor y electrónica. Lograr el funcionamiento del lazo PID.	Todo el equipo

6.2. Matriz de responsabilidades

En la siguiente matriz se indica con R al responsable principal de cada bloque y con C a los integrantes que colaboraron en él.

Tabla 5. Matriz de responsabilidades

Bloque	Garelli Valentín	Maggi Tomás	Rondón Juan Ignacio
Cuerpo del clinostato	C	C	R
Firmware	R	C	C
Hardware	C	R	C
Integración total	R	R	R

6.3. Dependencias entre bloques

Los bloques de cuerpo, firmware y hardware se desarrollaron en paralelo, ya que sus tareas son mayormente independientes. La integración total depende de los tres: requiere el cuerpo armado con el motor y el sensor montados, la etapa de potencia y la adaptación de niveles conectadas, y el firmware capaz de actuar sobre el motor y leer la velocidad. Por ello, la integración se ubica como la última etapa de la prueba de concepto, en concordancia con el enfoque secuencial adoptado.

7. Arquitectura del sistema

7.1. Diagrama de bloques general

El sistema se compone de el microcontrolador, los actuadores (motores) y el sensado de estado (velocidad). El microcontrolador cierra el lazo de control a partir de la velocidad medida por el sensor.

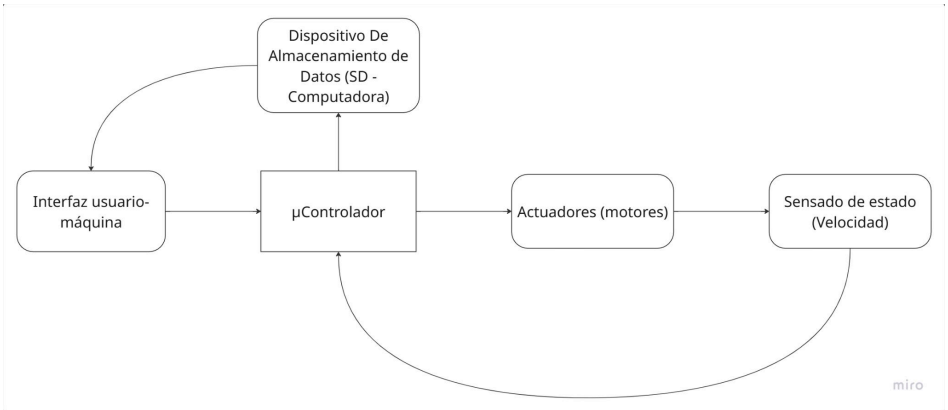


Figura 2. *Diagrama de bloques general del sistema*

7.2. Interfaces

Tabla 7. Interfaces del sistema.

Interfaz	Origen	Destino	Tipo
I01	MCU	Actuadores (puente H)	PWM
I02	Sensor (encoder)	MCU	Digital (pulsos, Input Capture)
I03	Actuadores	Sensor	Mecánico (rueda encoder)

8. Componentes principales

8.1. Microcontrolador: STM32 Nucleo-G491RE

El microcontrolador ejecuta la lógica de control y la toma de datos del dispositivo. Se utiliza la placa de desarrollo STM32 Nucleo-G491RE, que permite generar señales PWM de forma precisa y determinista mediante sus temporizadores, dispone de canales de captura de entrada (Input Capture) para la medición de señales, incorpora DMA y cuenta con unidad de punto flotante (FPU), lo que favorece la velocidad de cálculo del controlador y, a futuro, de las matrices de giro.



Figura 3. *Placa STM32G491RE*

8.2. Cuerpo y estructura

El elemento más relevante de la estructura son los marcos que rotan para generar el efecto de microgravedad. Se parte de un modelo diseñado para impresión 3D proveniente de un repositorio público (github.com/wahabk/Clinostat), pensado originalmente para dos motores paso a paso con transmisión por correas, que fue modificado para alojar los motores DC y el sensor.

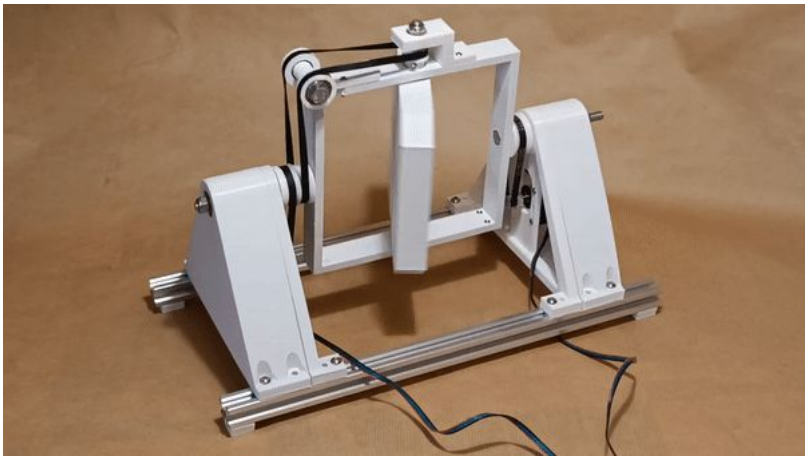


Figura 4. *Clinostato open-source imprimible en 3D.*

8.3. Motores DC JGA25-370

Se utilizan dos motores DC JGA25-370 con caja reductora para generar el movimiento en los dos ejes. En esta etapa se controla uno de ellos. Para satisfacer la demanda de corriente que solicita (con el microcontrolador no es posible) utilizamos una placa puente H (L298n).



Figura 5. *Motor DC*



Figura 6. *Motor DC*

8.4. Sensado de estado: encoder óptico

Para el sensado de velocidad del motor se utiliza un encoder óptico compuesto por dos elementos. El primero es una rueda solidaria al eje, con ventanas de igual tamaño equiespaciadas en su perímetro. El segundo es una placa que contiene un emisor infrarrojo, un fotodetector y un comparador. El emisor permanece encendido y apunta al fotodetector: cuando una ventana de la rueda deja pasar el haz, la salida pasa a nivel alto (uno lógico), y cuando el haz es interrumpido, la salida pasa a nivel bajo (cero lógico). De esta forma, al girar el motor se obtiene un tren de pulsos cuya frecuencia es proporcional a la velocidad de giro.

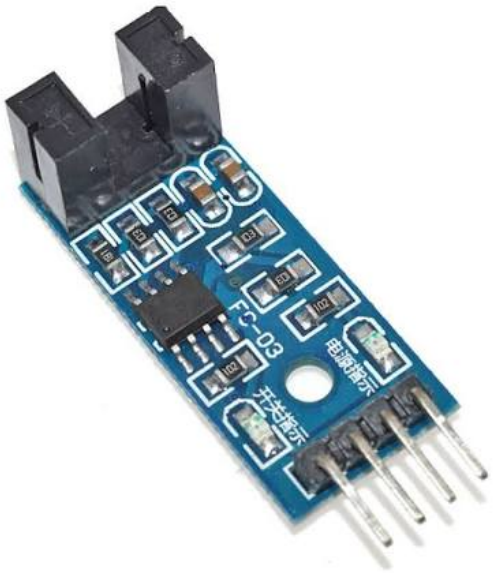


Figura 7. *Placa con fotodiodo y laser*



Figura 8. *Rueda de encoder*

9. Descripción técnica del sistema de control

9.1. Actuación: control del motor por PWM

El motor se acciona mediante una señal PWM generada por uno de los temporizadores del microcontrolador. La frecuencia de la señal se mantiene fija y lo que se modifica es su ciclo de trabajo

(duty cycle): al variarlo, se varía la tensión media aplicada al motor y, en consecuencia, su velocidad de giro. La señal PWM no se aplica directamente al motor, sino a través de un driver de tipo puente H, que entrega la corriente necesaria y permite además seleccionar el sentido de giro.

Frecuencia de PWM utilizada: 200Hz Resolución del duty cycle: 100

9.2. Sensado: medición de velocidad con timer

La velocidad se mide a partir de la señal del encoder utilizando un temporizador del microcontrolador configurado en modo de captura de entrada (Input Capture). Cada flanco de la señal del encoder dispara una captura del valor del contador del timer; la diferencia entre dos capturas consecutivas permite conocer el tiempo transcurrido entre ranuras. A partir de ese tiempo, de la frecuencia del timer y de la cantidad de ranuras de la rueda, se obtiene la velocidad de giro mediante el siguiente cálculo:

$$RPM = \frac{60^* 1000}{30 * \Delta t}$$

Frecuencia timer = 500 Hz

9.3. Lazo de control PID

Con la velocidad medida como realimentación, el microcontrolador calcula el error respecto de la velocidad de referencia y aplica un controlador PID. La salida del controlador se traduce en el ciclo de trabajo de la señal PWM, cerrando así el lazo de control sobre el eje sensado.

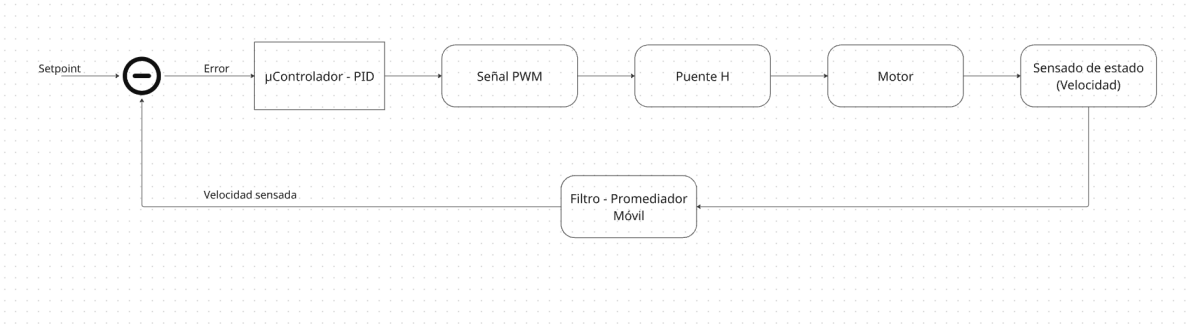


Figura 9. *Lazo de control*

10. Desarrollo de las tareas por bloque

En esta sección se detalla el desarrollo técnico de las tareas del bloque, incluyendo el diseño, los cálculos, las decisiones tomadas, los problemas encontrados y los resultados obtenidos.

10.1. Bloque 1: Cuerpo del clinostato

10.1.1. Modificación del cuerpo para los nuevos motores

El primer problema con el que nos encontramos al intentar armar el cuerpo fue la disposición de los motores seleccionados para el proyecto (motores DC), esto era algo que esperábamos puesto que el diseño del que nos basamos estaba preparado para otro tipo de motores. Para salvar este inconveniente propusimos tres soluciones:

1. *Rediseño de la base:*

Esta primera solución consistía en tomar las medidas de la base que sostenía a los motores para realizar las modificaciones necesarias utilizando una aplicación de diseño en 3D. La modificación consistía en agrandar el círculo central donde iban encastrados los motores de forma tal que cupiesen con los que contábamos nosotros. El desarrollo de está solución tomó aproximadamente 3-4 días de trabajo.

Luego de realizar las primeras pruebas con el nuevo diseño nos topamos con otro inconveniente que no habíamos percibido al principio. El cuerpo del que nos basamos no tenía un marco circular del que sostener a los motores, contaba con un marco ovalado de forma tal que si necesitábamos una mayor distancia vertical desde el eje del motor hasta la polea donde se ubicaba el otro lado la correa bastaba simplemente con un desajustar los tornillos que lo sostienen y ajustarlos un poco más abajo. El diseño que realizamos nosotros no contaba con esto, era simplemente un orificio del tamaño del motor centrado en altura de la base. Esto generaba que, al no estar perfectamente calculada la distancia, la correas patinara sobre la polea a causa de falta de tensión.

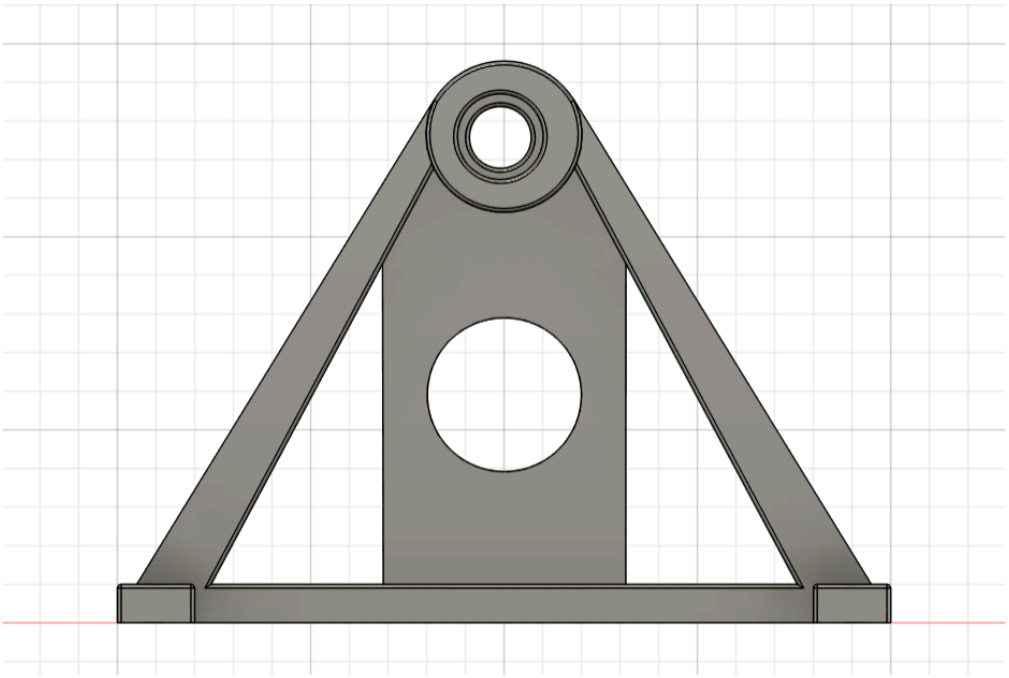


Figura 10. *Modificación solución 1*

2. *Aplanar la base:*

Con la finalidad de solucionar el inconveniente de la tensión de la polea planteamos una solución más “empírica”. Esta solución consistía en modificar una vez más la base donde iba colocado el

motor pero esta vez para que la superficie donde este encastraba quedase completamente plana. La idea era medir la distancia en el cuerpo original, el cual nos permitía ese juego en altura necesario para estirar la correa, y luego extrapolarlo a un orificio (realizado con torno) sobre el nuevo cuerpo plano del diámetro necesario para que el motor cupiese.

El inconveniente de la solución es que al ser de prueba y error, requería de bastantes iteraciones. El inconveniente de las iteraciones no sería el tiempo que tardamos en realizar las pruebas sino más bien la cantidad de cuerpos que quedaban desperdiciados (una vez modificado ya no podía modificarse nuevamente). Debido a esto y a que en 3 iteraciones no conseguimos nada puesto que necesitábamos una distancia que se regule mejor, la solución fue descartada.

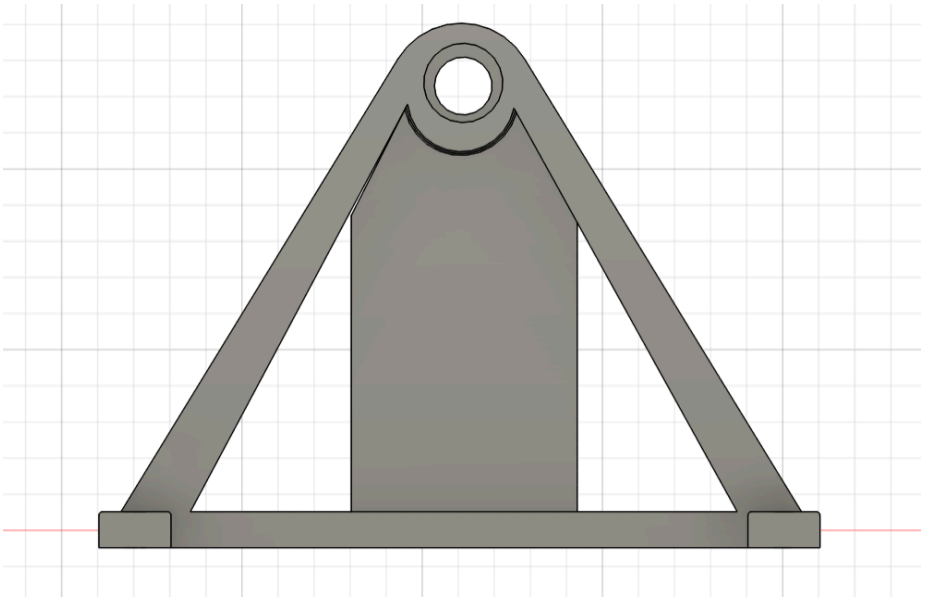


Figura 11. *Modificación solución 2*

3. Adaptar el motor:

Finalmente la última solución fue la que nos sirvió para satisfacer el principal problema que era adaptar el cuerpo para nuestros motores. La solución consiste en un diseño realizado en 3D cuadrado de 42.3mmX42.3mmX8mm, en las esquinas del cuadrado cuatro perforaciones para tornillos M3 y en el centro de este el orificio donde encastrarían los motores. De este modo, utilizando la base original podíamos acomodar la altura del motor a necesidad. Esto nos permitió lograr la tensión necesaria para las correas.

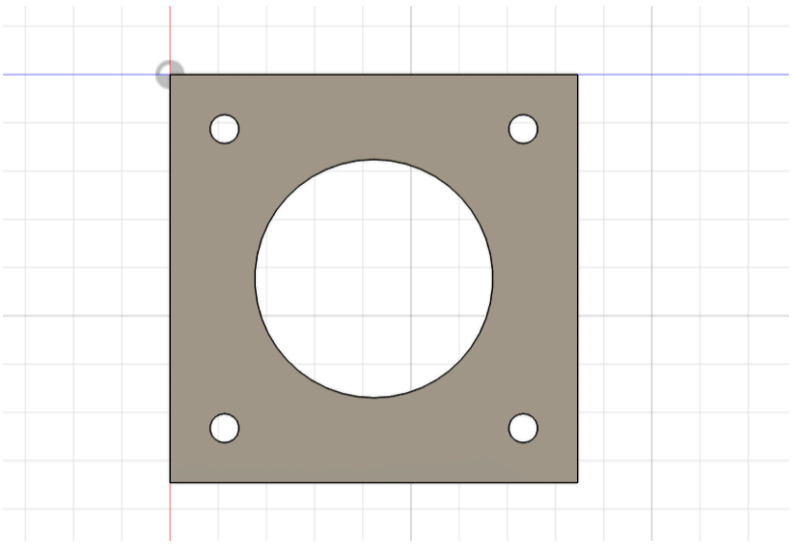


Figura 12. *Modificación solución 3*

10.1.2. Modificación del cuerpo para el sensor

La versión original del cuerpo no cuenta con un soporte para el sensor que utilizamos en el proyecto (ni siquiera trabaja con encoder), por lo que recurrimos a nuevas modificaciones sobre la estructura para mantener estático el sensor a la altura requerida conforme al diámetro de la rueda del encoder.

Para lograr esto la solución es sencilla. Diseñamos en 3D una especie de “estantería” que va atornillada a la base donde se encuentra el motor. El diseño a continuación:

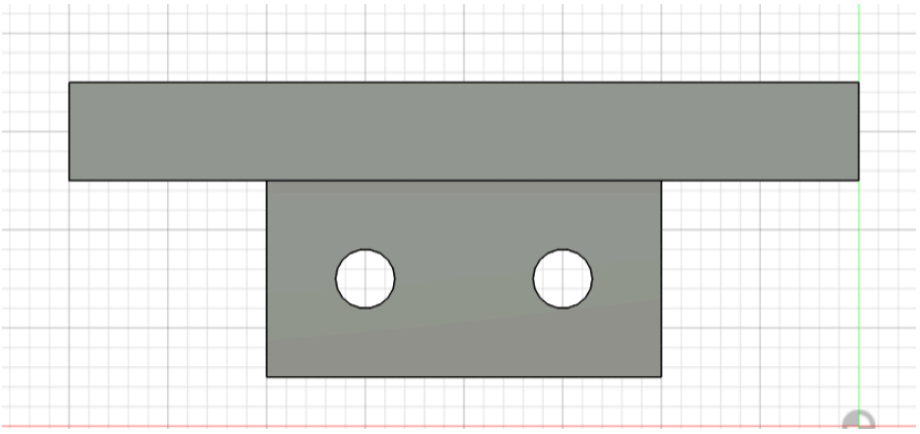


Figura 13. *Soporte sensor visto de frente*

10.1.3. Elevación del cuerpo

Para la elevación del cuerpo recurrimos a dos recortes de madera de igual altura y tamaño donde este va atornillado. Es necesario elevar en altura el cuerpo puesto que uno de sus ejes tiene una varilla que roza con el suelo.

10.2. Bloque 2: Firmware

10.2.1. Movimiento de los motores con PWM

Para la elección de la frecuencia del pwm testeamos, a través de un osciloscopio y un generador de funciones, distintos valores de frecuencia con una misma onda cuadrada, decidimos utilizar 200 Hz ya que era la frecuencia más alta que no presentaba deformación a la salida del **L298N**. Con las frecuencia definida se decidió utilizar un registro de comparación y captura (CCR) con un valor de 100 para que el valor del duty cycle tenga una correlación directa con el valor del CCR. En un momento barajamos la necesidad de utilizar un valor mayor de captura para tener mayor precisión en el porcentaje del duty cycle pero descartamos esta idea rápidamente ya que no presentaba un mayor nivel de control sobre la velocidad del motor.
Para lograr el movimiento de los motores decidimos implementar una librería de autoría propia la cual posee una función que indica el duty cycle que debe tener el pwm.

10.2.2. Sensado con el encoder mediante Input Capture

Para medir la velocidad angular de nuestro motor utilizamos una placa comercial, esta placa posee un LM393 con un sensor de luz. También utilizamos un encoder circular con 30 ventanas que permiten, o no, el paso de la luz, por lo que junto a la placa pudimos sensar el momento en el cual una ventana se encontraba frente al sensor de luz.
Al obtener un pulso cada vez que el sensor detectaba una ventana pudimos utilizar un timer en modo **Input Capture** para poder medir el tiempo relativo de dos ventanas respecto al inicio del timer, con esto, pudimos medir el tiempo transcurrido entre una ventana y otra. Al tener el tiempo transcurrido entre ventanas es posible calcular la velocidad en RPMs a la que se movía el motor a través de una fórmula matemática.
El primer intento de medir la velocidad fue fallido ya que utilizamos un vector que se sobreescribía constantemente por lo que a medida que se ejecutaba el programa al estar el vector dentro de una interrupción este se cambiaba y quedaban capturas más recientes en el lugar de las más antiguas. Luego de debuggear y encontrar el problema se cambio la lógica de captura para llegar al código final

```
if (htim->Instance == TIM2) {
    if (htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1) {

        uint32_t captura_actual_1 = HAL_TIM_ReadCapturedValue(htim,
        TIM_CHANNEL_1);

        if (primera_captura_1) {
            captura_anterior_1 = captura_actual_1;
            primera_captura_1 = 0;
            return; //sale del if si solo tengo una medicion y no puedo calcular delta
        }
        delta_m1 = captura_actual_1 - captura_anterior_1;
        captura_anterior_1 = captura_actual_1;
        flag_sensado_m1 = 1;
        tiempo_parado = 0;
```

Figura 14. *Callback del IC*

10.2.3. Implementación del control PID

Para la implementación del PID creamos una función la cual necesita los parámetros: Las RPM de set point y el motor que se desea controlar. Se utiliza un timer para llamar la función de manera periódica cada **10 ms**. Los diez milisegundos pasan a ser la base de tiempo del PID que, al ser constante, simplifica el cálculo del control ID. Por otra parte contamos con los valores de ganancias **Kp**,**Ki** y **Kd** los cuales se utilizan para sintonizar el PID para que responda a nuestras necesidades (luego de ser sintonizado las ganancias quedan fijas). En nuestro caso las ganancias que se utilizaron principalmente fueron Kp y Ki con valores de 4 y 1 respectivamente, la ganancia Kd quedó igualada a cero ya que nuestro proyecto no debe ni puede tener cambios bruscos de velocidad.

10.3. Bloque 3: Hardware

10.3.1. Adaptación de alimentación del sensor

La alimentación del sensor requiere de 5v pero solo contamos con una fuente de alimentación de 6v por lo que fue necesario adaptar los niveles de voltaje a través del uso de un divisor resistivo y un buffer.

El primer intento del circuito fue utilizando dos resistencias calculadas para obtener 5v entre medio de ellas (un divisor resistivo). Del punto medio de las resistencias conectamos una etapa buffer realizada con amplificadores operacionales para que la tensión del nodo deseado no varíe con la conexión del otro circuito. Esto terminó dando un resultado pobre debido a la incertidumbre en los valores de las resistencias. Las mismas variaban lo suficiente para ver un cambio notable en el voltaje de salida. Esta idea fue descartada y terminamos optando por utilizar un potenciómetro entre 6v y 0v con el pin del medio conectado al buffer. Luego de calibrar el circuito a través de los potenciómetros pudimos obtener un valor de voltaje de 5v fiable.

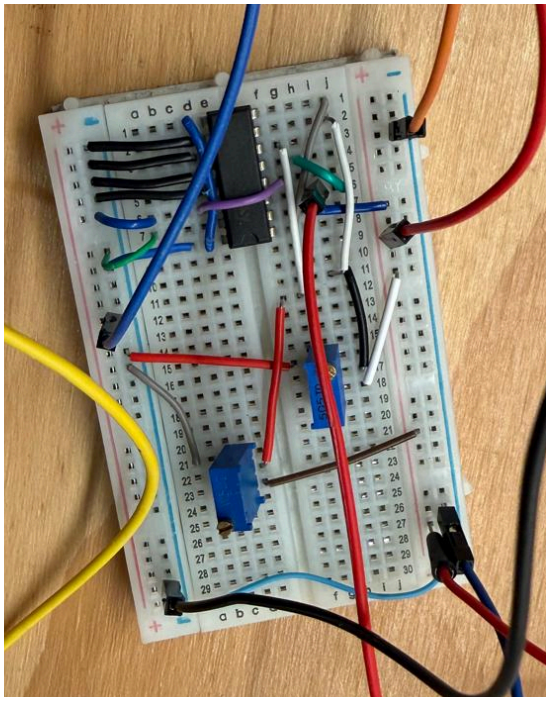


Figura 15. *Circuito de adaptación de alimentación*

10.3.2. Conexión del driver del motor (puente H)

El puente H tiene 7 conexiones relevantes para nuestra aplicación: la alimentación, los tres pines de control y la salida. La alimentación se eligió en 6v ya que es la alimentación máxima del motor, con el Duty Cycle de la señal PWM al 100% el motor se vería alimentado con 6v teniendo así rango completo de operación. El microcontrolador está conectado a los pines **IN1** e **IN2**, con esto se logra que, colocando en **UNO** lógico la salida conectada en **IN1** y en **CERO** lógico la conectada en **IN2**, el motor gire en un sentido y haciendo lo inverso gire en el sentido opuesto . Luego conectamos el PWM generado por la núcleo al pin **ENABLE** del puente H, con esto logramos controlar el motor en ambos sentidos utilizando una única señal **PWM**. Por último, la salida va directamente al motor indistintamente de polaridad ya que simplemente girará en dirección contraria.

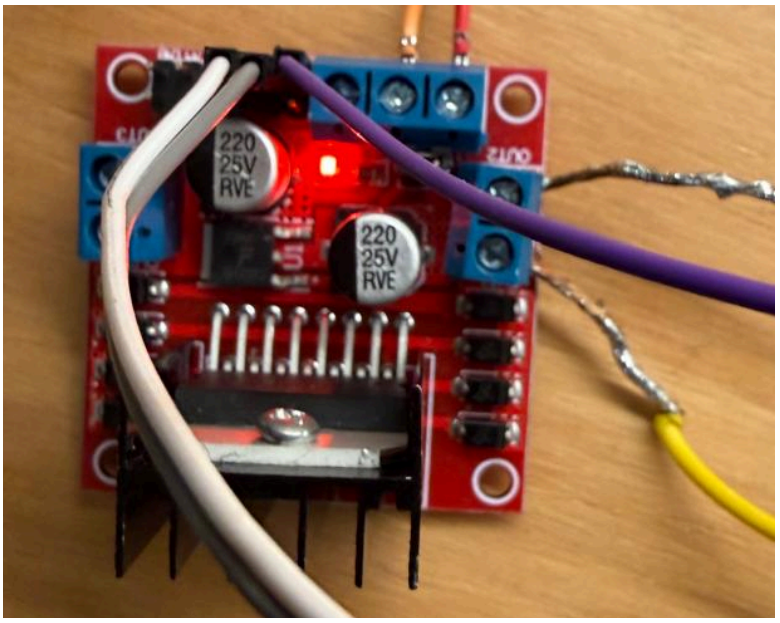


Figura 16. *Puente H*

Enumerando las funcionalidades:

- cable violeta: **Enable**
- cable gris: **IN1**
- cable blanco **IN2**
- cable rojo: **Alimentación (6v)**
- cable naranja: **GND**
- cables amarillos: **Salida**

10.4. Bloque 4: Integración total

Una de las últimas tareas que llevamos a cabo fue la integración del cuerpo en su totalidad. Una vez definidas y desarrolladas las tareas de los bloques anteriores, cada una logrando su objetivo y adaptándose a sus limitaciones, fue posible realizar la integración final, primero del cuerpo, luego de todo el sistema.

10.4.1. Armado del cuerpo

Para la integración del cuerpo se recurrió a la guía de armado proporcionada por el autor del repositorio sobre el que nos basamos para la estructura. Armada la base, proseguimos con el

ensamblado de las partes diseñadas por nosotros; atornillando la adaptación de los motores de tal forma que la tensión en la correa sea la suficiente para lograr un movimiento suave del marco y que no patinase sobre la polea, luego, atornillamos el estante sobre el que reposa el sensor de velocidad. Por último, elevamos el cuerpo sobre las maderas. El resultado es el siguiente:

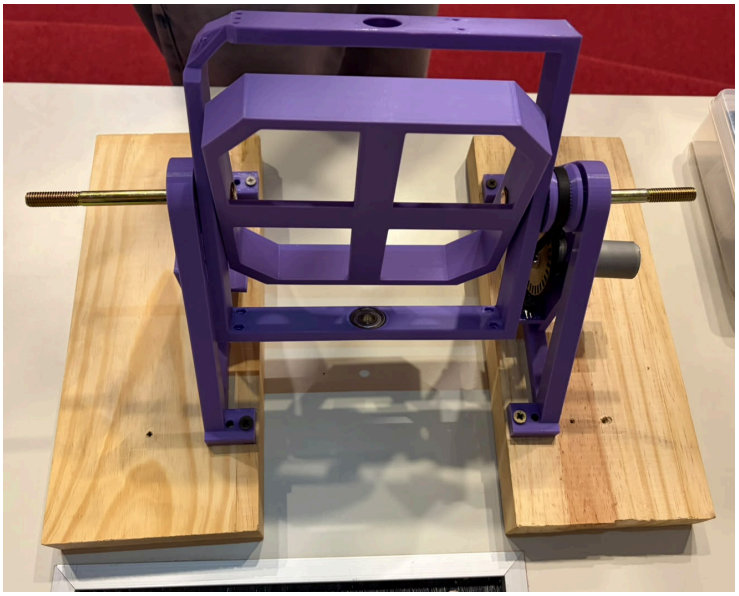


Figura 17. *Estructura armada*

10.4.2. Funcionamiento del lazo PID

El funcionamiento del bloque PID descrito a continuación:
Primero calculamos nuestro error el cual es la diferencia entre nuestros **SetPoint** (SP) y nuestra velocidad actual en **RPM**; la velocidad es nuestra variable de control. Con este error se puede aproximar qué tan lejos está nuestra velocidad actual de nuestro objetivo.

$$Error = SP - RPM$$

El cálculo anterior si bien sirve para aproximar nuestro motor a la velocidad deseada no tiene en cuenta el error de estado estacionario, el cual tiene efecto una vez que el sistema paso el estado transitorio. Para calcular este error lo que hicimos fue acumular en el tiempo el error calculado con cada iteración y multiplicarlo por la diferencia de tiempo entre el error anterior y el actual.

$$Integral = Acumulacion + Error * \Delta tiempo$$

El cálculo de la parte derivativa se decidió omitir ya que por la naturaleza lenta de las aplicaciones de nuestro clinostato no solo no podía ocurrir sino que tampoco debería, ya que los cambios bruscos de velocidad podrían arruinar ensayos.

Para el cálculo final de la salida nuestra fórmula queda compuesta de la manera siguiente:

$$Salida = KP * error + KI * integral$$

siendo KP y KI valores escalares los cuales se ajustan manualmente para obtener la respuesta deseada en nuestro PID. Los valores finales son KP = 4 y KI = 1.
Para ajustar los valores lo que hicimos fue sintonizar el valor KP hasta que la respuesta oscilara mientras que el valor KI era igual a cero. Luego aumentamos gradualmente KI hasta llegar a una velocidad aceptable para nuestra aplicación.

11. Resultados

A modo de prueba, y para mostrar el funcionamiento, colocamos el **"SetPoint"** en 15 RPM. Los resultados se encuentran dentro de lo esperado, pudimos lograr un tiempo de establecimiento de **20s** con un error en estado estacionario con un rango de **±0.01 rpm**. Debido a la naturaleza lenta de nuestro sistema estos valores son más que suficientes.

xpression	Type	Value
ccr_actual	uint32_t	57
rpm1	volatile float	15.000001
set_point	float	15
tiempo_pid	uint32_t	20712
error	volatile float	0.00749492645
error_3s	volatile float	6.66666698
error_5s	volatile float	4.68393326
error_10s	volatile float	1.90528107

Figura 18. *Resultados en el programa*

Podemos observar en la **figura 18** que iniciando con un pwm con un duty cycle de 0% pudimos llegar en 20 segundos a las 15 rpm esperadas. Además en la captura se puede observar como el error va disminuyendo en el tiempo viendo su magnitud a los 3.5 y 10 segundos inicializado el programa, mientras que la variable tiempo _pid indica la cantidad de milisegundos que transcurrieron hasta que la diferencia entre el setpoint y las rpm medidas fuera menor a 0.01.

Con esto se demuestra el correcto funcionamiento del proyecto desarrollado. Es importante haber obtenido estos resultados puesto que, como dijimos anteriormente, la idea principal de este proyecto es que funcionase como prueba de concepto de la tesis que esperamos desarrollar posteriormente.